Mikroprocesorová technika

Prednáška č.4: Aplikačné využitie časovačov a prerušení

prof. Ing. Ivan Virgala, PhD.

Technická univerzita v Košiciach
Strojnícka fakulta

Katedra priemyselnej automatizácie a mechatroniky
ARM-lab (Applied Robotics and Mechatronics Laboratory)



Aplikácie prerušení a časovačov

Funkcie v jazyku C, časové meranie a obsluha udalostí

Tematické vymedzenie

Nadväznosť na časovače a prerušenia

- organizácia programu pomocou funkcií v jazyku C,
- meranie časového intervalu pomocou hardvérového časovača,
- spracovanie hrán vstupného signálu pomocou prerušenia,
- aplikácia na ultrazvukový snímač vzdialenosti,
- meranie frekvencie externého periodického signálu.

Použitá platforma

Arduino UNO R3 s mikrokontrolérom `ATmega328P`, programovanie v jazyku C bez Arduino frameworku, priamy prístup k registrom periférií.

Dva typy meracích úloh

Meranie dĺžky impulzu

- sleduje sa začiatok a koniec impulzu,
- časovač meria čas medzi dvoma hranami,
- typická aplikácia: ultrazvukový snímač.

Meranie frekvencie

- počítajú sa opakujúce sa hrany signálu,
- časovač určuje dĺžku meracieho okna,
- typická aplikácia: signál z generátora alebo inkrementálneho snímača.

Funkcie v jazyku C

Rozdelenie programu na menšie časti

Definícia funkcie v jazyku C

```c
uint16_t max2(uint16_t a, uint16_t b)
{
    if (a > b) {
        return a;
    }
    else {
        return b;
    }
}
```

Časti funkcie

- `uint16_t` – návratový typ,
- `max2` – názov funkcie,
- `a`, `b` – formálne parametre,
- `return` – odovzdanie výsledku volajúcemu kódu.

Syntax

Hlavička funkcie sa neukončuje bodkočiarkou, pretože za ňou nasleduje telo funkcie v zložených zátvorkách.

Volanie funkcie a vyhodnotenie parametrov

```
1  uint16_t sucet3(uint16_t a,
2                  uint16_t b,
3                  uint16_t c)
4  {
5      return a + b + c;
6  }
7
8  int main(void)
9  {
10     uint16_t x;
11     uint16_t c = 13;
12
13     x = sucet3(5, c, 2*c);
14 }
```

Princíp

Pri volaní funkcie sa najprv vyhodnotia výrazy v zátvorke.

V tomto prípade funkcia dostane hodnoty:

$$5, \quad 13, \quad 26$$

a výsledkom je:

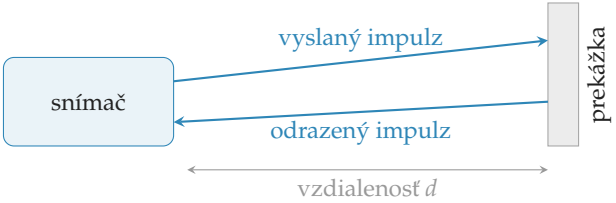
$$x = 44.$$

Ultrazvukový snímač SRF05

Spúšťací impulz, signál ECHO a prepočet času na vzdialenosť

Princíp ultrazvukového merania vzdialenosti

Ultrazvukový snímač vyšle krátky akustický impulz, ktorý sa odrazí od prekážky a vráti späť k prijímaču. Meraná je doba prechodu signálu tam aj späť.



Základný vzťah

$$d = \frac{v \cdot t}{2}$$

kde v je rýchlosť zvuku a t je celkový čas prechodu signálu k prekážke a späť.

Pin snímača	Funkcia
`VCC`	napájanie snímača 5 V
`GND`	spoločná zem so systémom mikrokontroléra
`TRIG`	vstup snímača; krátky impulz spustí merací cyklus
`ECHO`	výstup snímača; šírka impulzu zodpovedá dobe letu ultrazvuku

Rozsah použitia

Merací rozsah snímača SRF05 približne 1 cm až 4 m.

Časový priebeh signálov TRIG a ECHO

Spúšťací impulz na vstupe `TRIG` vyvolá meranie. Následne snímač nastaví výstup `ECHO` do logickej jednotky po dobu zodpovedajúcu letu ultrazvukového impulzu.



Meraná veličina

Mikrokontrolér nemeria priamo vzdialenosť. Meria čas trvania impulzu `ECHO` a z neho vypočíta vzdialenosť.

Prepočet šírky impulzu na vzdialenosť

Pri teplote okolo 20 °C je rýchlosť zvuku približne 343 m/s. V praxi sa pre ultrazvukové moduly často používa zjednodušený prepočet:

$$d_{cm} \approx \frac{t_{echo,\mu s}}{58}$$

Príklad

Ak má impulz `ECHO` šírku 1160 µs, potom:

$$d \approx \frac{1160}{58} = 20\,\text{cm}$$

Faktor 2

Nameraný čas zahŕňa cestu signálu od snímača k prekážke aj späť. Pri výpočte vzdialenosti sa preto pracuje s polovicou celkovej dráhy.

Funkcia ako oddelenie výpočtu od obsluhy udalosti

```
1 uint16_t vzdialenost_cm(uint16_t ticks)
2 {
3     return ticks / 116;
4 }
```

Význam

Ak $f_{CPU} = 16\,\text{MHz}$ s N = 8, takže register časovača sa inkrementuje každých 0,5 μs. Preto:

$$t_{us} = 0{,}5 \cdot ticks$$

$$d_{cm} \approx \frac{t_{us}}{58}$$

$$d_{cm} \approx \frac{ticks}{116}$$

Organizácia kódu

- ISR zachytí udalosť a uloží nameranú hodnotu,
- výpočet vzdialenosti sa vykoná mimo ISR,
- samostatná funkcia zjednoduší opakované použitie výpočtu.

Poznámka

ISR má zostať krátka; výpočty a rozhodovanie patria do hlavnej slučky programu.

Input Capture časovača Timer1

Princíp

`Input Capture` je hardvérová funkcia časovača `Timer1`, ktorá pri hrane signálu na pine `PB0/ICP1` automaticky uloží aktuálnu hodnotu časovača `TCNT1` do registra `ICR1`.

Meranie impulzu ECHO

- nábežná hrana uloží čas začiatku impulzu,
- zostupná hrana uloží čas konca impulzu,
- rozdiel hodnôt určí šírku impulzu,
- z času impulzu sa vypočíta vzdialenosť.

Výhoda

Zachytenie času vykoná hardvér priamo v okamihu hrany. Program potom iba spracuje uloženú hodnotu v registri `ICR1`.

Dôležité

`PB0/ICP1` nie je `INT0`. Ide o samostatný vstup Input Capture časovača `Timer1`.

Signál	ATmega328P	Arduino UNO	Poznámka
`TRIG`	`PB1`	D9	výstup, vytvorenie krátkeho impulzu
`ECHO`	`PB0/ICP1`	D8	vstup pre hardvérové zachytenie času Timer1
`LED`	`PB5`	D13	indikačná LED na doske UNO

Dôvod voľby pinu `PB0/ICP1`

Funkcia `Input Capture` časovača Timer1 umožňuje uložiť hodnotu `TCNT1` presne v okamihu detegovanej hrany na vstupe `ICP1`.

Meranie času pomocou Timer1

Input Capture, hrany signálu ECHO a prepočet počtu taktov

Prečo je vhodný Timer1

Vlastnosti Timer1

- 16-bitový čítač/časovač,
- dostatočný rozsah pre impulz `ECHO`,
- vstup `ICP1` na hardvérové zachytenie času,
- prerušenie pri udalosti `Input Capture`.

Merací princíp

Na nábežnej hrane signálu `ECHO` sa uloží začiatočný čas. Na zostupnej hrane sa uloží koncový čas. Rozdiel týchto hodnôt je šírka impulzu.

Preddelička časovača a rozlíšenie merania

Preddelička delí hodinový signál mikrokontroléra pred vstupom do časovača. Určuje, ako rýchlo sa zvyšuje hodnota registra `TCNT1`.

Preddelička	**Frekvencia časovača pri** 16 MHz	**Dĺžka jedného kroku**
1	16 MHz	62.5 ns
8	2 MHz	0.5 µs
64	250 kHz	4 µs
256	62.5 kHz	16 µs
1024	15.625 kHz	64 µs

Voľba pre meranie ECHO

Pri preddeličke 8 má jeden krok časovača trvanie 0.5 µs. Maximálny rozsah 16-bitového časovača je približne 32.8 ms, čo postačuje pre meranie vzdialenosti do niekoľkých metrov.

Výpočet času z hodnoty časovača

Ak Timer1 pracuje s preddeličkou 8 pri takte 16 MHz, jedna jednotka časovača zodpovedá 0.5 µs.

$$t_{echo,\mu s} = \frac{N}{2}$$

Príklad

Ak rozdiel zachytených hodnôt je $N = 2320$, potom:

$$t_{echo} = 1160\,\mu s$$

$$d \approx \frac{1160}{58} = 20\,cm$$

Význam symbolu N

N je rozdiel počtu krokov časovača medzi nábežnou a zostupnou hranou signálu `ECHO`.

Input Capture v Timer1

Funkcia `Input Capture` uloží aktuálnu hodnotu `TCNT1` do registra `ICR1` v okamihu vybranej hrany na vstupe `ICP1`.



Prepínanie detegovanej hrany

Po zachytení nábežnej hrany sa nastavenie zmení na zostupnú hranu, aby sa zaznamenal koniec impulzu `ECHO`.

Registre Timer1 použité pri meraní

Register/bit	Funkcia v meraní
`TCNT1`	aktuálna hodnota 16-bitového časovača
`ICR1`	hodnota `TCNT1` uložená pri hrane na vstupe `ICP1`
`TCCR1B`	nastavenie zdroja hodín, preddeličky a výberu hrany `ICES1`
`TIMSK1`	povolenie prerušenia `ICIE1` pri udalosti Input Capture
`TIFR1`	príznakový register udalostí Timer1

Inicializácia Timer1 pre Input Capture

```
void timer1_capture_init(void)
{
    TCCR1A = 0;
    TCCR1B = 0;
    TCNT1  = 0;

    TCCR1B |= (1 << ICES1);  // najprv nabezna hrana
    TCCR1B |= (1 << CS11);   // preddelicka 8
    TIMSK1 |= (1 << ICIE1);  // prerusenie Input Capture
}
```

Poznámka k nastaveniu

Bit `ICES1` určuje, ktorá hrana na vstupe `ICP1` vyvolá zachytenie hodnoty `TCNT1`. Bity `CS12..CS10` určujú zdroj hodín a preddeličku.

Implementačná štruktúra merania

Funkcie, prerušenia a hlavná slučka programu

Generovanie spúšťacieho impulzu TRIG

```
1  #define F_CPU 16000000UL
2  #include <avr/io.h>
3  #include <util/delay.h>
4
5  void srf05_trigger(void)
6  {
7      PORTB &= ~(1 << PB1);
8      _delay_us(2);
9      PORTB |= (1 << PB1);
10     _delay_us(10);
11     PORTB &= ~(1 << PB1);
12 }
```

Funkcia v programe

Funkcia vytvorí krátky impulz na výstupe `TRIG`. Po tomto impulze snímač vykoná merací cyklus a na výstupe `ECHO` vytvorí impulz s dĺžkou žodpovedajúcou vzdialenosti.

Obsluha prerušenia Input Capture

```
volatile uint16_t echo_start = 0;
volatile uint16_t echo_ticks = 0;
volatile uint8_t  echo_ready = 0;
volatile uint8_t  cakam_na_koniec = 0;

ISR(TIMER1_CAPT_vect)
{
    uint16_t capture = ICR1;

    if (!cakam_na_koniec) {
        echo_start = capture;
        cakam_na_koniec = 1;
        TCCR1B &= ~(1 << ICES1);  // dalsia je zostupna hrana
    } else {
        echo_ticks = capture - echo_start;
        echo_ready = 1;
        cakam_na_koniec = 0;
        TCCR1B |= (1 << ICES1);   // dalsia je nabezna hrana
    }
}
```

Interpretácia obsluhy zachytenia hrán

Prvá zachytená hrana

- predstavuje začiatok impulzu `ECHO`,
- hodnota `ICR1` sa uloží ako `echo_start`,
- detekcia sa prepne na zostupnú hranu.

Druhá zachytená hrana

- predstavuje koniec impulzu `ECHO`,
- rozdiel hodnôt určí šírku impulzu,
- nastaví sa príznak `echo_ready` pre hlavnú slučku.

Obsluha prerušenia Input Capture

```
1  volatile uint16_t echo_start = 0;
2  volatile uint16_t echo_ticks = 0;
3  volatile uint8_t echo_ready = 0;
4
5  ISR(TIMER1_CAPT_vect)
6  {
7      uint16_t capture = ICR1;
8
9      if (TCCR1B & (1 << ICES1)) {
10         // nabezna hrana
11         echo_start = capture;
12
13         // dalsia: zostupna hrana
14         TCCR1B &= ~(1 << ICES1);
15     }
16     else {
17         // zostupna hrana
18         echo_ticks = capture - echo_start;
19         echo_ready = 1;
20
21         // dalsia: nabezna hrana
22         TCCR1B |= (1 << ICES1);
23     }
24 }
```

Interpretácia

- Hrana na `ICP1` uloží aktuálnu hodnotu časovača do `ICR1`.
- Pri **nábežnej hrane** sa hodnota uloží do `echo_start`.
- Detekcia sa prepne na **zostupnú hranu**.
- Pri zostupnej hrane sa vypočíta dĺžka impulzu:
- `echo_ready = 1` signalizuje, že meranie je dokončené.
- Detekcia sa opäť nastaví na nábežnú hranu.

Prepočet nameraných taktov na centimetre

Pri preddeličke 8 a takte 16 MHz má jeden krok Timer1 hodnotu 0.5 µs. Najprv sa teda z počtu taktov získa čas impulzu v mikrosekundách.

```
uint16_t distance_cm_from_ticks(uint16_t ticks)
{
    uint16_t echo_us = ticks / 2;
    return echo_us / 58;
}
```

Výpočtová funkcia

Meranie času a prepočet na vzdialenosť sú oddelené. Zmena preddeličky alebo iný prepočet vzdialenosti sa potom upravuje iba v tejto funkcii.

Inicializácia Timer1 pre meranie šírky impulzu

```
1  void timer1_init(void)
2  {
3      TCCR1A = 0;
4
5      TCCR1B = (1 << ICES1) |
6               (1 << CS11);
7
8      TCNT1 = 0;
9
10     TIFR1 = (1 << ICF1);
11
12     TIMSK1 = (1 << ICIE1);
13 }
```

Nastavenie Timer1

- `TCCR1A = 0` – normálny režim časovača,
- `ICES1 = 1` – prvá zachytená udalosť bude **nábežná hrana** na `ICP1`,
- `CS11 = 1` – preddelička 8,
- `TCNT1 = 0` – vynulovanie čítača,
- `ICF1` – vymazanie prípadného starého príznaku zachytenia,
- `ICIE1 = 1` – povolenie prerušenia Input Capture.

Inicializácia vstupno-výstupných pinov

```
1  void gpio_init(void)
2  {
3      // TRIG - PD4 ako vystup
4      DDRD |= (1 << PD4);
5      PORTD &= ~(1 << PD4);
6
7      // ECHO - PB0/ICP1 ako vstup
8      DDRB &= ~(1 << PB0);
9      PORTB &= ~(1 << PB0);
10 }
```

Konfigurácia pinov

- PD4 je pripojený na vstup **TRIG** snímača,
- Bit PD4 v registri DDRD nastavíme ako výstup,
- Výstup PD4 je po inicializácii v logickej 0,
- PB0/ICP1 je pripojený na výstup **ECHO** snímača,
- Bit PB0 v registri DDRB nastavíme ako vstup,
- Pull-up rezistor na PB0 vypnutý.

Hlavná slučka programu

```c
int main(void)
{
    gpio_init();
    timer1_capture_init();
    sei();

    while (1) {
        srf05_trigger();
        _delay_ms(60);

        if (echo_ready) {
            uint16_t ticks = echo_ticks;
            echo_ready = 0;

            uint16_t d_cm = distance_cm_from_ticks(ticks);
            // spracovanie nameranej vzdialenosti
        }
    }
}
```

Prenos informácie medzi ISR a hlavnou slučkou

Príznak dokončeného merania

Premenná `echo_ready` signalizuje, že obsluha prerušenia zachytila obe hrany impulzu `ECHO` a hodnota `echo_ticks` obsahuje šírku impulzu v krokoch Timer1.

ISR

- vykoná iba rýchle uloženie času,
- vypočíta rozdiel taktov,
- nastaví príznak dokončeného merania.

Hlavná slučka

- zistí nastavený príznak,
- prepočíta hodnotu na vzdialenosť,
- vykoná aplikačné rozhodovanie.

Typické situácie

- neprišla zostupná hrana signálu `ECHO`,
- nameraný impulz je mimo očakávaného rozsahu,
- vznikol odraz od nevhodného povrchu alebo mimo osi snímača,
- merania nasledujú príliš rýchlo po sebe.

Implementačné riešenie

Merací cyklus má mať definovaný maximálny čas čakania. Ak sa v tomto intervale meranie nedokončí, výsledok sa označí ako neplatný a spustí sa ďalší merací cyklus.

Filtrácia merania vzdialenosti

Jednotlivé meranie ultrazvukovým snímačom môže byť ovplyvnené odrazom, geometriou prekážky alebo rušením. Aplikačný program preto zvyčajne pracuje s upravenou hodnotou.

Jednoduché prístupy

- odmietnutie hodnoty mimo rozsahu,
- priemerovanie niekoľkých meraní,
- medián z malého počtu vzoriek,
- obmedzenie maximálnej zmeny medzi vzorkami.

Význam pre riadenie

Stabilnejšia hodnota vzdialenosti je vhodnejšia pre rozhodovanie, signalizáciu alebo jednoduchú reguláciu. Filtrácia však zvyšuje oneskorenie merania.

Meranie frekvencie externého signálu

Počítanie impulzov v definovanom časovom okne

Meranie frekvencie externého periodického signálu

Frekvencia sa určí z počtu zachytených hrán N počas presne definovaného meracieho okna T_g:

$$f = \frac{N}{T_g}.$$

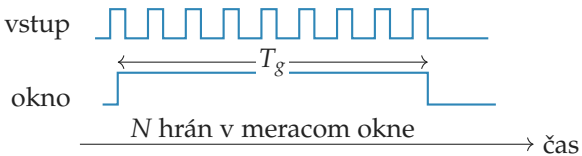
Vstupný signál

- každá sledovaná hrana zvýši čítač impulzov,
- hrany môže zachytávať externé prerušenie,
- pri vysokých frekvenciách je vhodnejší hardvérový čítač.

Meracie okno

- časovač vytvorí interval T_g,
- po jeho uplynutí sa uloží počet impulzov,
- počítadlo sa vynuluje a začne nové meranie.

Časový priebeh merania frekvencie



Vyhodnotenie

Počas meracieho okna T_g sa počíta počet zachytených hrán N.

$$f = \frac{N}{T_g}$$

Príklad

Pri $T_g = 1\,\text{s}$ platí: počet impulzov v okne je priamo frekvencia v hertzoch.

Rozdelenie úloh medzi externé prerušenie a časovač

Externé prerušenie

- vstupný signál je privedený napríklad na `PD2/INT0`,
- každá nábežná hrana zvýši počet impulzov,
- ISR má obsahovať iba krátku inkrementáciu počítadla.

Časovač

- vytvára presné meracie okno,
- po uplynutí okna uloží počet impulzov,
- vynuluje počítadlo pre ďalšie okno.

Dôležité

Pri vyšších frekvenciách môže byť externé prerušenie príliš často vyvolávané. Vtedy je vhodné využiť hardvérový vstup čítača, napríklad `T1` pri `Timer/Counter1`.

Počítanie hrán vstupného signálu

```
1  volatile uint16_t pulse_count = 0;
2  volatile uint16_t freq_hz = 0;
3  volatile uint8_t  freq_ready = 0;
4  ISR(INT0_vect)
5  {
6      pulse_count++;
7  }
8  ISR(TIMER1_COMPA_vect)
9  {
10     freq_hz = pulse_count;
11     pulse_count = 0;
12     freq_ready = 1;
13 }
```

Význam premenných

- `pulse_count` – počet hrán v okne,
- `freq_hz` – posledný výsledok,
- `freq_ready` – príznak pre hlavnú slučku.

Interpretácia

Pri okne 1 s je `freq_hz` frekvencia v Hz.

Režim CTC časovača

Princíp

Režim `CTC` znamená *Clear Timer on Compare Match*. Časovač počíta od nuly po hodnotu uloženú v registri `OCR1A`. Keď nastane zhoda `TCNT1 = OCR1A`, časovač sa automaticky vynuluje.

Použitie

- vytvorenie presného periodického časového intervalu,
- nastavenie dĺžky intervalu pomocou registra `OCR1A`,
- možnosť vyvolať prerušenie pri každej zhode.

$$T = \frac{(OCR1A + 1) \cdot \text{preddelička}}{f_{\text{CPU}}}$$

Pri $F_{CPU} = 16\,\text{MHz}$ a preddeličke 1024 má časovač frekvenciu:

$$\frac{16\,000\,000}{1024} = 15625\,\text{Hz}.$$

Pre meracie okno 1 s nastavíme v režime CTC hodnotu `OCR1A`=15624.

```
1 TCCR1B = (1 << WGM12) | (1 << CS12) | (1 << CS10);
2 OCR1A  = 15624;
3 TIMSK1 = (1 << OCIE1A);
```

Poznámka

Časovač počíta od 0 po hodnotu `OCR1A`. Počet krokov je preto 15624 + 1 = 15625.

Spracovanie výsledku v hlavnej slučke

```c
int main(void)
{
    // konfigurácia INT0 a Timer1

    while (1) {
        if (freq_ready) {
            freq_ready = 0;

            uint16_t f = freq_hz;
            // zobrazenie, rozhodovanie,
            // regulácia alebo diagnostika
        }
    }
}
```

Architektúra

- `INT0` vytvára počet impulzov,
- `Timer1` určuje časové okno,
- hlavná slučka spracuje hotový výsledok.

Metóda počítania impulzov a metóda merania periódy

Počítanie impulzov v okne

- jednoduchá implementácia,
- vhodné pre stredné a vyššie frekvencie,
- rozlíšenie závisí od dĺžky meracieho okna.

Meranie periódy

- meria sa čas medzi dvoma hranami,
- vhodné pre nízke frekvencie,
- výhodné je použitie `Timer1` a funkcie Input Capture.

Voľba metódy

Pri nízkych frekvenciách je presnejšie merať čas medzi hranami. Pri vyšších frekvenciách je výhodné počítať počet impulzov počas pevne zvoleného meracieho intervalu.

Integrovaná meracia aplikácia

Od hardvérovej udalosti k spracovanému výsledku

Porovnanie realizovaných meraní

Meranie	Použité udalosti	Výsledná veličina
Ultrazvukový snímač	začiatok a koniec impulzu `ECHO`	vzdialenosť
Meranie frekvencie	počet hrán v časovom okne	frekvencia signálu

Spoločný princíp

Prerušenie zachytí časovo významnú udalosť, časovač poskytne časovú referenciu a hlavná slučka vykoná výpočet alebo ďalšie rozhodovanie.

Odporúčaná štruktúra programu

- obslužné rutiny prerušení majú byť krátke,
- spoločné premenné medzi ISR a hlavným programom majú byť označené `volatile`,
- viacbajtové výsledky treba čítať konzistentne,
- výpočty a rozhodovanie je vhodné vykonávať mimo ISR,
- pri meraní treba ošetriť aj časový limit a neplatný stav signálu.

Ďakujem za pozornosť